EXP NO - 04: SYSTEM CALLS

```
[rene@localhost ~]$ mkdir system_calls
[rene@localhost ~]$ ls
abc.sh     ch         f.c           memory     Pictures      Templates
a.out      Desktop    fcfs.c        Music      pid.c         text
bank       Documents  filename.txt  name.sh    producer.c    text.c
bankers    Downloads  file.sh       newfile.c  Public        Videos
bankers.c  e.sh       firstfit.c    page       rec           xyz
cd         exec.c     first.sh      p.c        system_calls
[rene@localhost ~]$ cd system_calls
[rene@localhost system_calls]$ vi pid.c
```

```
#include <stdio.h>
#include <unistd.h>
int main(){
        printf("PID: %d\n",getpid());
        printf("PPID: %d\n",getppid());
        return 0;
}
~
~
~
```

```
[rene@localhost ~]$ mkdir system_calls
[rene@localhost ~]$ ls
abc.sh     ch          f.c           memory     Pictures      Templates
a.out      Desktop     fcfs.c        Music      pid.c         text
bank       Documents   filename.txt  name.sh    producer.c    text.c
bankers    Downloads   file.sh       newfile.c  Public        Videos
bankers.c  e.sh        firstfit.c    page       rec           xyz
cd         exec.c      first.sh      p.c        system_calls
[rene@localhost ~]$ cd system_calls
[rene@localhost system_calls]$ vi pid.c
[rene@localhost system_calls]$ gcc pid.c
[rene@localhost system_calls]$ ./a.out
PID: 2759
PPID: 2687
[rene@localhost system_calls]$
```

```
[rene@localhost ~]$ cd system_calls
[rene@localhost system_calls]$ vi fork.c
```

```c
#include<stdio.h>
#include<unistd.h>
int main()
{
        if(fork()==0){
                printf("This is CHILD process\n");
        }else{
                printf("This is Parent process\n");
        }
        printf("PID: %d\n", getpid());
        return 0;
}
```

```
"fork.c" 13L, 198B                                                                    13,0-1        All
```

```
[rene@localhost ~]$ cd system_calls
[rene@localhost system_calls]$ vi fork.c
[rene@localhost system_calls]$ gcc fork.c
[rene@localhost system_calls]$ ./a.out
This is Parent process
PID: 2709
This is CHILD process
PID: 2710
[rene@localhost system_calls]$
```

```
[rene@localhost system_calls]$ vi exec.c
[rene@localhost system_calls]$
```

```c
#include<stdio.h>
#include<unistd.h>
int main(){
        printf("Before exec()\n");
        execl("/bin/ls","ls",NULL);
        printf("After exec()\n");
        return 0;
}

~
~
~
~
~
~
~
~
~
~
~
~
~
```

```
[rene@localhost system_calls]$ vi exec.c
[rene@localhost system_calls]$ gcc exec.c
[rene@localhost system_calls]$ ./a.out
Before exec()
a.out  exec.c  fork.c  pid.c
[rene@localhost system_calls]$
```

```c
#include<stdio.h>
#include<dirent.h>
int main(){
        DIR *dir;
        dir = opendir(".");
        if(dir == NULL){
                printf("Failed to open directory\n");
                return 1;
        }
        printf("Directory opened sucessfully!\n");
        closedir(dir);
        return 0;
}
```

"opendir.c" 14L, 228B 14,0-1 All

```
rene@localhost system_calls]$ vi opendir.c
rene@localhost system_calls]$ gcc opendir.c
rene@localhost system_calls]$ ./a.out
irectory opened sucessfully!
rene@localhost system_calls]$
```

```
[rene@localhost system_calls]$ vi opendir.c
[rene@localhost system_calls]$ gcc opendir.c
[rene@localhost system_calls]$ ./a.out
Directory opened sucessfully!
[rene@localhost system_calls]$ vi readdir.c
```

```
#include<stdio.h>
#include<dirent.h>
int main(){
        DIR *dir;
        struct dirent *entry;
        dir = opendir(".");
        if(dir == NULL){
                printf("Cannot open directory\n");
                return 1;
        }
        printf("Files in directory:\n");
        while((entry=readdir(dir)) != NULL){
                printf("%s\n", entry->d_name);
        }
        closedir(dir);
        return 0;
}
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
"readdir.c" 17L, 311B                                                                                    17,1          All
```

```
[rene@localhost system_calls]$ vi readdir.c
[rene@localhost system_calls]$ gcc readdir.c
[rene@localhost system_calls]$ ./a.out
Files in directory:
.
..
pid.c
fork.c
exec.c
opendir.c
readdir.c
a.out
```